

BEE 4750 Lab 1: Julia and GitHub Basics

Name: Moho Goswami

ID: mg2297

git config --global user.name "Mona Lisa" git config --global user.email "monalisa@gmail.com"

Due Date

Thursday, 8/28/25, 9:00pm

Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
Activating project at `c:\Users\mohog\lab1-mohogos`
  Updating registry at `C:\Users\mohog\.julia\registries\General.toml`
  Updating `C:\Users\mohog\lab1-mohogos\Project.toml`
[7073ff75] + IJulia v1.30.1
[b964fa9f] + LaTeXStrings v1.4.0
[91a5bcdd] + Plots v1.40.19
  Updating `C:\Users\mohog\lab1-mohogos\Manifest.toml`
[66dad0bd] + AliasTables v1.1.3
[d1d4a3ce] + BitFlags v0.1.9
[944b1d66] + CodecZlib v0.7.8
[35d6a980] + ColorSchemes v3.30.0
[3da002f7] + ColorTypes v0.12.1
```

[c3611d14] + ColorVectorSpace v0.11.0
[5ae59095] + Colors v0.13.1
[f0e56b4a] + ConcurrentUtilities v2.5.0
[8f4d0f93] + Conda v1.10.2
[d38c429a] + Contour v0.6.3
[9a962f9c] + DataAPI v1.16.0
[864edb3b] + DataStructures v0.19.1
[8bb1440f] + DelimitedFiles v1.9.1
[ffbed154] + DocStringExtensions v0.9.5
[460bff9d] + ExceptionUnwrapping v0.1.11
[c87230d0] + FFMPEG v0.4.4
[53c48c17] + FixedPointNumbers v0.8.5
[1fa38f19] + Format v1.3.7
[28b8d3ca] + GR v0.73.17
[42e2da0e] + Grisu v1.0.2
[cd3eb016] + HTTP v1.10.17
[7073ff75] + IJulia v1.30.1
[92d709cd] + IrrationalConstants v0.2.4
[1019f520] + JLFzf v0.1.11
[692b3bcd] + JLLWrappers v1.7.1
[682c06a0] + JSON v0.21.4
[b964fa9f] + LaTeXStrings v1.4.0
[23fbe1c1] + Latexify v0.16.9
[2ab3a3ac] + LogExpFunctions v0.3.29
[e6f89c97] + LoggingExtras v1.1.0
[1914dd2f] + MacroTools v0.5.16
[739be429] + MbedTLS v1.1.9
[442fdcdd] + Measures v0.3.2
[e1d29d7a] + Missings v1.2.0
[77ba4419] + NaNMath v1.1.3
[4d8831e6] + OpenSSL v1.5.0
[bac558e1] + OrderedCollections v1.8.1
[69de0a69] + Parsers v2.8.3
[ccf2f8ad] + PlotThemes v3.3.0
[995b91a9] + PlotUtils v1.4.3
[91a5bcdd] + Plots v1.40.19
[aea7be01] + PrecompileTools v1.2.1
[21216c6a] + Preferences v1.5.0
[43287f4e] + PtrArrays v1.3.0
[3cdcf5f2] + RecipesBase v1.3.4
[01d81517] + RecipesPipeline v0.6.12
[189a3867] + Reexport v1.2.2
[05181044] + RelocatableFolders v1.0.1

[ae029012] + Requires v1.3.1
 [6c6a2e73] + Scratch v1.3.0
 [992d4aef] + Showoff v1.0.3
 [777ac1f9] + SimpleBufferStream v1.2.0
 [a2af1166] + SortingAlgorithms v1.2.2
 [860ef19b] + StableRNGs v1.0.3
 [10745b16] + Statistics v1.11.1
 [82ae8749] + StatsAPI v1.7.1
 [2913bbd2] + StatsBase v0.34.6
 [62fd8b95] + TensorCore v0.1.1
 [3bb67fe8] + TranscodingStreams v0.11.3
 [5c2747f8] + URIs v1.6.1
 [1cfade01] + UnicodeFun v0.4.1
 [1986cc42] + Unitful v1.24.0
 [45397f5d] + UnitfulLatexify v1.7.0
 [41fe7b60] + Unzip v0.2.0
 [81def892] + VersionParsing v1.3.0
 [c2297ded] + ZMQ v1.4.1
 [6e34b625] + Bzip2_jll v1.0.9+0
 [83423d85] + Cairo_jll v1.18.5+0
 [ee1fde0b] + Dbus_jll v1.16.2+0
 [2702e6a9] + EpollShim_jll v0.0.20230411+1
 [2e619515] + Expat_jll v2.7.1+0
 [b22a6f82] + FFMPEG_jll v7.1.1+0
 [a3f928ae] + Fontconfig_jll v2.17.1+0
 [d7e528f0] + FreeType2_jll v2.13.4+0
 [559328eb] + FriBidi_jll v1.0.17+0
 [0656b61e] + GLFW_jll v3.4.0+2
 [d2c73de3] + GR_jll v0.73.17+0
 [b0724c58] + GettextRuntime_jll v0.22.4+0
 [7746bdde] + Glib_jll v2.84.3+0
 [3b182d85] + Graphite2_jll v1.3.15+0
 [2e76f6c2] + HarfBuzz_jll v8.5.1+0
 [aacddb02] + JpegTurbo_jll v3.1.2+0
 [c1c5ebd0] + LAME_jll v3.100.3+0
 [88015f11] + LERC_jll v4.0.1+0
 [1d63c593] + LLVMOpenMP_jll v18.1.8+0
 [dd4b983a] + LZ0_jll v2.10.3+0
 [e9f186c6] + Libffi_jll v3.4.7+0
 [7e76a0d4] + Libglvnd_jll v1.7.1+1
 [94ce4f54] + Libiconv_jll v1.18.0+0
 [4b2f31a3] + Libmount_jll v2.41.1+0
 [89763e89] + Libtiff_jll v4.7.1+0

[38a345b3] + Libuuid_jll v2.41.1+0
 [e7412a2a] + Ogg_jll v1.3.6+0
 [458c3c95] + OpenSSL_jll v3.5.2+0
 [91d4177d] + Opus_jll v1.5.2+0
 [36c8627f] + Pango_jll v1.56.3+0
 [30392449] + Pixman_jll v0.44.2+0
 [c0090381] + Qt6Base_jll v6.8.2+1
 [629bc702] + Qt6Declarative_jll v6.8.2+1
 [ce943373] + Qt6ShaderTools_jll v6.8.2+1
 [e99dba38] + Qt6Wayland_jll v6.8.2+1
 [a44049a8] + Vulkan_Loader_jll v1.3.243+0
 [a2964d1f] + Wayland_jll v1.24.0+0
 [ffd25f8a] + XZ_jll v5.8.1+0
 [f67eecfb] + Xorg_libICE_jll v1.1.2+0
 [c834827a] + Xorg_libSM_jll v1.2.6+0
 [4f6342f7] + Xorg_libX11_jll v1.8.12+0
 [0c0b7dd1] + Xorg_libXau_jll v1.0.13+0
 [935fb764] + Xorg_libXcursor_jll v1.2.4+0
 [a3789734] + Xorg_libXdmcp_jll v1.1.6+0
 [1082639a] + Xorg_libXext_jll v1.3.7+0
 [d091e8ba] + Xorg_libXfixes_jll v6.0.1+0
 [a51aa0fd] + Xorg_libXi_jll v1.8.3+0
 [d1454406] + Xorg_libXinerama_jll v1.1.6+0
 [ec84b674] + Xorg_libXrandr_jll v1.5.5+0
 [ea2f1a96] + Xorg_libXrender_jll v0.9.12+0
 [c7cfdc94] + Xorg_libxcb_jll v1.17.1+0
 [cc61e674] + Xorg_libxkbfile_jll v1.1.3+0
 [e920d4aa] + Xorg_xcb_util_cursor_jll v0.1.5+0
 [12413925] + Xorg_xcb_util_image_jll v0.4.1+0
 [2def613f] + Xorg_xcb_util_jll v0.4.1+0
 [975044d2] + Xorg_xcb_util_keysyms_jll v0.4.1+0
 [0d47668e] + Xorg_xcb_util_renderutil_jll v0.3.10+0
 [c22f9ab0] + Xorg_xcb_util_wm_jll v0.4.2+0
 [35661453] + Xorg_xkbcomp_jll v1.4.7+0
 [33bec58e] + Xorg_xkeyboard_config_jll v2.44.0+0
 [c5fb5394] + Xorg_xtrans_jll v1.6.0+0
 [8f1865be] + ZeroMQ_jll v4.3.6+0
 [3161d3a3] + Zstd_jll v1.5.7+1
 [35ca27e7] + eudev_jll v3.2.14+0
 [214eeab7] + fzf_jll v0.61.1+0
 [a4ae2306] + libaom_jll v3.12.1+0
 [0ac62f75] + libass_jll v0.17.4+0
 [1183f4f0] + libdecor_jll v0.2.2+0

[2db6ffa8] + libevdev_jll v1.13.4+0
[f638f0a6] + libfdk_aac_jll v2.0.4+0
[36db933b] + libinput_jll v1.28.1+0
[b53b4c65] + libpng_jll v1.6.50+0
[a9144af2] + libsodium_jll v1.0.21+0
[f27f6e37] + libvorbis_jll v1.3.8+0
[009596ad] + mtdev_jll v1.1.7+0
[1270edf5] + x264_jll v10164.0.1+0
[dfaa095f] + x265_jll v4.1.0+0
[d8fb68d0] + xkbcommon_jll v1.9.2+0
[0dad84c5] + ArgTools v1.1.2
[56f22d72] + Artifacts v1.11.0
[2a0f44e3] + Base64 v1.11.0
[ade2ca70] + Dates v1.11.0
[f43a241f] + Downloads v1.6.0
[7b1f6079] + FileWatching v1.11.0
[b77e0a4c] + InteractiveUtils v1.11.0
[b27032c2] + LibCURL v0.6.4
[76f85450] + LibGit2 v1.11.0
[8f399da3] + Libdl v1.11.0
[37e2e46d] + LinearAlgebra v1.11.0
[56ddb016] + Logging v1.11.0
[d6f4376e] + Markdown v1.11.0
[a63ad114] + Mmap v1.11.0
[ca575930] + NetworkOptions v1.2.0
[44cfe95a] + Pkg v1.11.0
[de0858da] + Printf v1.11.0
[3fa0cd96] + REPL v1.11.0
[9a3f8284] + Random v1.11.0
[ea8e919c] + SHA v0.7.0
[9e88b42a] + Serialization v1.11.0
[6462fe0b] + Sockets v1.11.0
[2f01184e] + SparseArrays v1.11.0
[f489334b] + StyledStrings v1.11.0
[fa267f1f] + TOML v1.0.3
[a4e569a6] + Tar v1.10.0
[8dfed614] + Test v1.11.0
[cf7118a7] + UUIDs v1.11.0
[4ec0a83e] + Unicode v1.11.0
[e66e0078] + CompilerSupportLibraries_jll v1.1.1+0
[deac9b47] + LibCURL_jll v8.6.0+0
[e37daf67] + LibGit2_jll v1.7.2+0
[29816b5a] + LibSSH2_jll v1.11.0+1

```

[c8fffd9c3] + MbedTLS_jll v2.28.6+0
[14a3606d] + MozillaCACerts_jll v2023.12.12
[4536629a] + OpenBLAS_jll v0.3.27+1
[05823500] + OpenLibm_jll v0.8.5+0
[efcefd7] + PCRE2_jll v10.42.0+1
[bea87d4a] + SuiteSparse_jll v7.7.0+0
[83775a58] + Zlib_jll v1.2.13+1
[8e850b90] + libblastrampoline_jll v5.11.0+0
[8e850ede] + nghttp2_jll v1.59.0+0
[3f19e933] + p7zip_jll v17.4.0+2

```

Info Packages marked with have new versions available but compatibility constraints
 Precompiling project...

```

Conda
IJulia
Plots → IJuliaExt

```

0 dependencies successfully precompiled in 20 seconds. 182 already precompiled.

The following 2 direct dependencies failed to precompile:

IJuliaExt

Failed to precompile IJuliaExt [2f4121a4-3b3a-5ce6-9c5e-1f2673ce168a] to "C:\\Users\\mohog\\
 ERROR: LoadError: IJulia not properly installed. Please run Pkg.build("IJulia")

Stacktrace:

```

[1] error(s::String)
   @ Base .\error.jl:35
[2] top-level scope
   @ C:\Users\mohog\.julia\packages\IJulia\8qH9K\src\IJulia.jl:52
[3] include
   @ .\Base.jl:562 [inlined]
[4] include_package_for_output(pkg::Base.PkgId, input::String, depot_path::Vector{String}, o
   @ Base .\loading.jl:2881
[5] top-level scope
   @ stdin:6

```

in expression starting at C:\Users\mohog\.julia\packages\IJulia\8qH9K\src\IJulia.jl:1

in expression starting at stdin:6

ERROR: LoadError: Failed to precompile IJulia [7073ff75-c697-5162-941a-fcdaad2a7d2a] to "C:\\

Stacktrace:

```

[1] error(s::String)
   @ Base .\error.jl:35
[2] compilecache(pkg::Base.PkgId, path::String, internal_stderr::IO, internal_stdout::IO, l
   @ Base .\loading.jl:3174
[3] (::Base.var"#1110#1111"{Base.PkgId})()

```

```

@ Base .\loading.jl:2579
[4] mkpidlock(f::Base.var"#1110#1111"{Base.PkgId}, at::String, pid::Int32; kwopts::@Kwargs{
@ FileWatching.Pidfile C:\Users\mohog\AppData\Local\Programs\Julia-1.11.6\share\julia\st
[5] #mkpidlock#6
@ C:\Users\mohog\AppData\Local\Programs\Julia-1.11.6\share\julia\stdlib\v1.11\FileWatchi
[6] trymkpidlock(::Function, ::Vararg{Any}; kwargs::@Kwargs{stale_age::Int64})
@ FileWatching.Pidfile C:\Users\mohog\AppData\Local\Programs\Julia-1.11.6\share\julia\st
[7] #invokelatest#2
@ .\essentials.jl:1057 [inlined]
[8] invokelatest
@ .\essentials.jl:1052 [inlined]
[9] maybe_cache_file_lock(f::Base.var"#1110#1111"{Base.PkgId}, pkg::Base.PkgId, srcpath::St
@ Base .\loading.jl:3698
[10] maybe_cache_file_lock
@ .\loading.jl:3695 [inlined]
[11] _require(pkg::Base.PkgId, env::Nothing)
@ Base .\loading.jl:2565
[12] __require_prelocked(uuidkey::Base.PkgId, env::Nothing)
@ Base .\loading.jl:2388
[13] #invoke_in_world#3
@ .\essentials.jl:1089 [inlined]
[14] invoke_in_world
@ .\essentials.jl:1086 [inlined]
[15] _require_prelocked
@ .\loading.jl:2375 [inlined]
[16] _require_prelocked
@ .\loading.jl:2374 [inlined]
[17] macro expansion
@ .\lock.jl:273 [inlined]
[18] require(uuidkey::Base.PkgId)
@ Base .\loading.jl:2371
[19] top-level scope
@ C:\Users\mohog\.julia\packages\Plots\xKhUG\ext\IJuliaExt.jl:6
[20] include
@ .\Base.jl:562 [inlined]
[21] include_package_for_output(pkg::Base.PkgId, input::String, depot_path::Vector{String},
@ Base .\loading.jl:2881
[22] top-level scope
@ stdin:6
in expression starting at C:\Users\mohog\.julia\packages\Plots\xKhUG\ext\IJuliaExt.jl:1
in expression starting at stdin:6
IJulia

```

```

Failed to precompile IJulia [7073ff75-c697-5162-941a-fcdaad2a7d2a] to "C:\\Users\\mohog\\.julia\\cache\\IJulia\\7073ff75-c697-5162-941a-fcdaad2a7d2a"
ERROR: LoadError: IJulia not properly installed. Please run Pkg.build("IJulia")
Stacktrace:
 [1] error(s::String)
   @ Base .\error.jl:35
 [2] top-level scope
   @ C:\Users\mohog\.julia\packages\IJulia\8qH9K\src\IJulia.jl:52
 [3] include
   @ .\Base.jl:562 [inlined]
 [4] include_package_for_output(pkg::Base.PkgId, input::String, depot_path::Vector{String}, depot_key::String)
   @ Base .\loading.jl:2881
 [5] top-level scope
   @ stdin:6
in expression starting at C:\Users\mohog\.julia\packages\IJulia\8qH9K\src\IJulia.jl:1
in expression starting at stdin:6

```

This next cell loads packages which are required for the rest of the code evaluation. In this case, we only need to load the `Plots.jl` plotting package, but you will see others over the course of the semester (and can add more if desired; just make sure that you’ve [added the new packages to the environment](#)). Standard Julia practice is to load all of the needed packages at the top of the file.

Warning

Loading packages can take a while, especially the first time! Julia tries to precompile all of the packages you’re using so repeat use is faster, but this can be quite slow at first.

```
using Plots
```

Introduction

Julia

Julia is an up-and-coming language, originally developed for scientific programming. While learning a new programming language always has its hiccups, the good news is that if you’ve programmed in a high-level language such as Python or MATLAB, most Julia concepts should look familiar.

If you have not successfully set up Julia, follow the instructions in [Tools Setup](#) and/or ask for help.

You can use other editors for this course, but our recommendation is [Visual Studio Code](#) with the [Julia extension](#), which will make life a *lot* simpler! You should have set this up by following the [Tools Setup](#) instructions, but if not, do so now and/or ask for help.

Jupyter Notebooks

Jupyter notebooks integrate text and equations in Markdown with Julia (or Python, or R) code. To do this, Jupyter notebooks consist of two types of “cells”: code cells and Markdown (text) cells.

Click once on this section of text. A box will appear around this text (and some areas above/below it) - all of that is within this cell.

Markdown is a text markup framework for formatting language that makes things look pretty when viewed across different platforms: web browsers, notebooks, and so forth. Text written in Markdown can also include hyperlinks, LaTeX equations, section headers, and images, among other features (most of [the course website](#) and the lecture notes were all written in Markdown!). [Here is a basic Markdown cheat sheet](#).

What you are looking at right now is the formatted text after the Markdown is processed. To see the raw Markdown, do one of:

- press **Enter** while that cell is selected, or
- double-click on that cell.

A couple of the features you will see in this Markdown cell:

- The --- command creates a horizontal line. This is also nice for separating sections.
- Backticks (``...``) can be used to format and highlight code, keystrokes, etc.
- The # sign is used to create a new section header; two # signs (##) is used to create a new subsection header; ### creates a subsubsection, and so on.
- You can create a bulleted list by using the asterisk * or a dash - and a space.
- You can create regular text by just typing as usual.
- You can create **bold-faced text** by wrapping it with two asterisks on both sides.
- You can create *italicized text* by wrapping it with a single asterisk on both sides.
- To create a new paragraph, you must include a blank line between the old and new paragraphs.

At this point you might be wondering how to turn this cell back into the fully formatted Markdown text instead of the raw Markdown you’re probably still looking at. You have a couple of options, depending on your platform, but the most consistent is to type **Shift + Enter** to **execute** the cell (this is also how to run code, but more on that later).

Additionally, you will frequently need to create new cells in your Jupyter notebooks. How you do this will depend on how you interact with the notebook, but try to figure this out now.

One tip is to think carefully about what bits of code should be in the same cell, as you typically only see output from the last command in a cell. For example, compare the following:

```
x = 5  
sin(x)
```

with

```
x = 5
```

5

```
sin(x)
```

-0.9589242746631385

In Julia, you can also suppress the output of a command with a semi-colon:

```
sin(x);
```

which can help if you want to split some code out for clarity or to insert some text prior to it, but don't want to clutter the notebook with its output.

For code cells, to execute the commands within the cell, we also press **Shift+Enter**.

Finally, **make sure that you evaluate all of the code cells in order before submitting**.

One bad outcome with notebooks occurs when cells are evaluated out of order, so fixed bugs and edits in previous cells do not get a chance to propagate down. You can do this with the **Run All** command in whichever interface you're using to edit your notebook.

Julia Basics

There are many tutorials and references for Julia, including a [basics overview on the class website](#). Please feel free to reference these as you work through any part of the course.

Formatting Math

It will often be helpful to include nicely-formatted mathematics in a notebook. Markdown accomodates this using LaTeX syntax. A LaTeX cheatsheet is available on the class website, and many other guides exist online.

Below is an example of a formatted equation:

$$x = 5.$$

Looking For Help

There is no shame in using Google, or other resources, for help when programming. There are many, many times when you can't quite get the syntax to work, can't quite figure out the right package or command to use, or are feeling too lazy or overwhelmed (I'm not judging either of those!) to dig through the documentation. Some good resources include:

- [Stack Overflow](#) is a treasure trove of answers;
- The [official Julia forum](#) and the [Julia Subreddit](#) are also very useful.

You are also highly encouraged to post on [Ed Discussion](#), though getting a response might be less immediate. Just be mindful that to get good answers, [you have to help people help you](#), and **make sure to give credit to any resources that were helpful!**

Exercises (3 points)

Use your understanding of Julia syntax and the GitHub workflow to complete the following (hopefully short) exercises. Convert your completed lab assignment to a PDF and submit it to the Gradescope Assignment "Lab 1".

Remember to:

- Include a (succinct but clear) writeup of the core idea underlying your code, through some combination of equations, text, and algorithms. As you are not required to submit your code, we will not be looking at it in detail, and instead will rely on those writeups to assess whether your approach is correct.
- If using the notebook, evaluate all of your code cells, in order (using a **Run All** command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

Computing a Dot Product

Given two numeric arrays `x` and `y`, write a function to compute their dot product if they have equal length, and return an error if not (this is useful for debugging!). Use the following code as a starting point. **Do not use a built-in function.**

```
function dot_product(x, y)
  if length(x) == length(y)
    output = 0
    for i in 1:length(x)
      output += x[i] * y[i]
    end
    return output
  else
    throw(DimensionMismatch("length of x not equal to length of y"))
  end
end
```

`dot_product` (generic function with 1 method)

I first determined the length of each array using `length()` and then checked if they were equal to each other using a double equal sign. Then, I calculated the dot product for the two arrays `x` and `y`.

Here are some tests to make sure your code works as intended. Tests like these are useful to make sure everything works as intended. One reason to split your code up into functions is that it makes it straightforward to write tests to make sure each piece of your code works, which makes it easier to identify where errors are occurring.

```
dot_product([1 2 3], [4 5 6])
```

If you know the value you should get, you can write a more formal test using the `@assert` macro, which is a good way to “automate” checking (since you get an obvious error if the code doesn’t work as desired):

```
@assert dot_product([1 2 3], [4 5 6]) == 32
```

```
UndefVarError: UndefVarError: `dot_product` not defined in `Main`
Suggestion: check for spelling errors or missing imports.
UndefVarError: `dot_product` not defined in `Main`
```

Suggestion: check for spelling errors or missing imports.

Stacktrace:

```
[1] top-level scope
```

```
@ c:\Users\mohog\lab1-mohogos\jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_X22sZmlsZ
```

Let's also make sure we get an error when the dimensions of the two vectors don't match:

```
dot_product([1 2 3], [4 5])
```

UndefVarError: UndefVarError: `dot\_product` not defined in `Main`

Suggestion: check for spelling errors or missing imports.

UndefVarError: `dot\_product` not defined in `Main`

Suggestion: check for spelling errors or missing imports.

Stacktrace:

```
[1] top-level scope
```

```
@ c:\Users\mohog\lab1-mohogos\jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_X24sZmlsZ
```

Making a Plot

Write a function to compute the square of an integer x . Evaluate this function for integers between $x = -5$ and $x = 5$ and make a plot of the squared values (you can find a quick guide to making various types of plots [here](#)). Make sure to label your axes.

```
using Plots
```

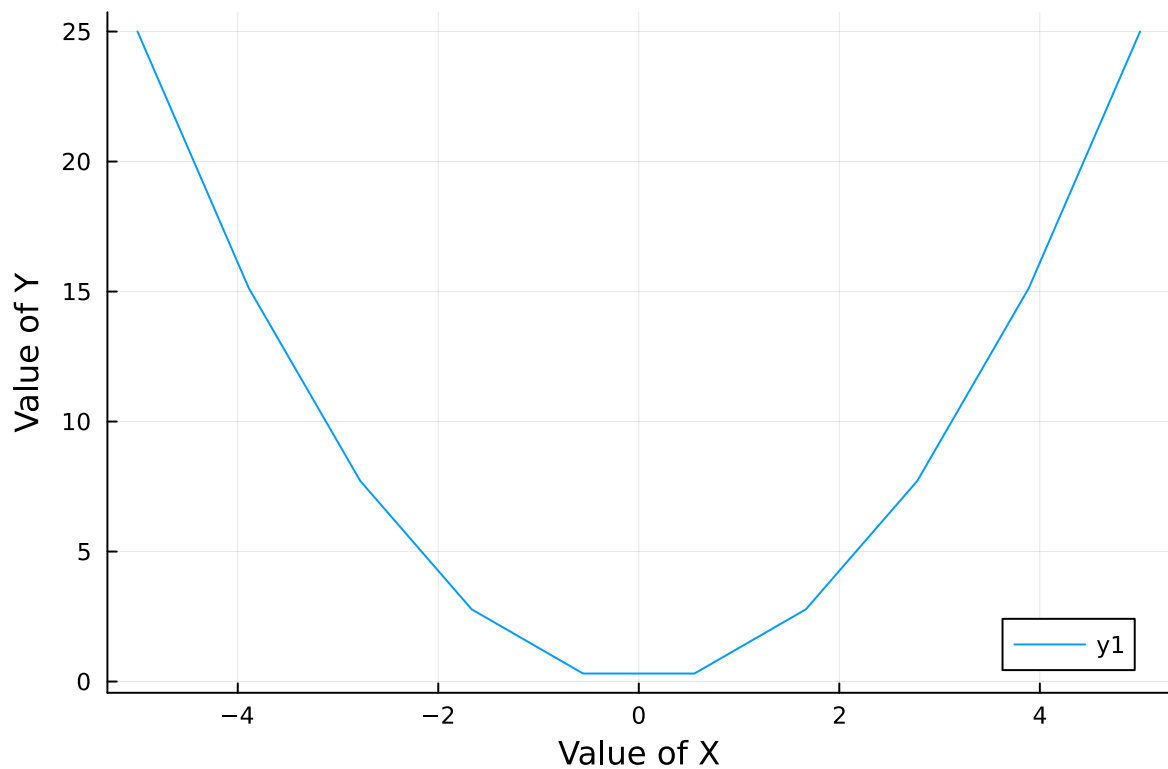
```
x = range(-5, 5, length = 10)
```

```
function squarex(x)
```

```
    return x^2
```

```
end

y = squarex.(x)
plot(x, y)
xlabel!("Value of X")
ylabel!("Value of Y")
```



I wrote the range for x as being from -5 to 5 with 10 plot points so that each integer value would be included, then wrote a function to define x-squared and set $y = x\text{-squared}$. Then, I plotted the x values (the initial given range) against the y values (the squared values of each x) and added labels.

Commit and Push Your Changes to GitHub

After completing the previous two exercises, commit your solution file (notebook or otherwise) and push to GitHub. Use an informative commit message which makes it clear what changes you've made. The specific workflow for this will vary depending on how you're writing up your solutions; please search for specifics and ask for help as needed!

Useful Commit Sizes

Ideally, you'd commit whenever you make a “substantial” enough change that you want to lock in, such as writing the core code for a problem or completing a problem, if you're preparing code to be used elsewhere (by yourself or others), or if you want to ask for help. `git` lets you revert changes back to a previous commit, so it's easy to undo changes or updates which broke something that was previously working, so changing too many things at once can make it hard to keep track of what worked when.

But in this case, go ahead and just commit after finishing the problems.

Push the repository with these commits to GitHub and take a screenshot of the repository page (<https://github.com/BEE4750-FA25/<username>/lab01>) which shows the updated repository. Include that screenshot in your submission as the solution to this problem.

Submitting PDF

Important

These submission instructions will not be repeated on future assignments!

Export your writeup as a PDF (this doesn't have to be a notebook if you're struggling to export: you can do your writeup in *e.g.* Microsoft Word) and submit it to the “Lab 1” assignment on Gradescope. **Make sure that you tag pages corresponding to relevant problems to avoid a 10% penalty.**

Printing Code to PDF

You are not required to submit your code when submitting assignments. However, when printing a notebook to PDF (whether you do this locally or via the GitHub Action), long lines will run off the edge of code cells, which may result in comments or code being hidden. If you see this, go back to the notebook and break up long lines into shorter ones (for example, see the comment in the above code cell) to ensure key parts of your results aren't missing.